

Measuring Validator Utility in Generate–Verify–Repair NetOps Pipelines: a Confirmatory Paired Study with Shared Initial Candidates

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory study (AC-2026-03) to evaluate whether a multidimensional validator metric suite predicts end-to-end agentic NetOps success better than a single verifier pass rate. Using a synthetic intent→configuration generator and a hosted generate–validate–repair (GVR) adapter under shared_initial_candidate semantics, we ran 200 paired tasks (one shared initial generation per task reused for baseline and guarded). The deterministic validator (deterministic_netops_validator_v5, v5.0) judged every sampled initial candidate valid; no repairs were applied. Baseline = 200/200, guarded = 200/200, paired difference = 0.00 (bootstrap 95% CI [0.00, 0.00], exact McNemar two-sided $p = 1.0$; bootstrap_seed = 1729, resamples = 10000). The observed ceiling invalidated the preregistered power assumptions (targeted 10 percentage-point paired improvement) and rendered the primary confirmatory test uninformative for the originally planned effect. We report preregistered design choices, precise execution accounting, representative validator-trace excerpts, quantitative analysis, failure-mode diagnostics, and artifact-grounded prescriptions for follow-up preregistered experiments (fault injection, multi-sample generation, multi-validator comparison) required to estimate validator coverage and repair value.

I. INTRODUCTION

Automating NetOps workflows with generative models requires reliable mechanisms to avoid harmful, silent, or wrong-but-plausible actions. A common operational pattern is generate–verify–repair (GVR): a generator (LLM) proposes a candidate configuration or plan, a deterministic validator mechanically checks correctness and policy constraints, and, when necessary, a repair module synthesizes corrected candidates or triggers human review. Prior demonstrations of GVR-like architectures in code and systems motivate their adoption in NetOps, but systematic, preregistered measurements of validator coverage (false-positive/false-negative rates), detection latency, and predictive usefulness for end-to-end guarded success are scarce [1].

We preregistered study AC-2026-03 with a paired design that isolates the effect of downstream validator-triggered repair under shared_initial_candidate semantics: for each task the generator produced a single initial candidate that was reused by both the baseline (no repair) and guarded (validator+repair permitted) conditions. Under these semantics any paired difference can only arise down-

stream from validator-triggered repair, not from independent generator sampling. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline and the preregistered confirmatory hypothesis (H1) expected that a multidimensional validator-metric suite would explain more variance in end-to-end success than a single verifier pass rate. Our main contribution is a fully preregistered, artifact-archived experiment and an exact, transparent report of a degenerate confirmatory outcome (ceiling) with concrete, artifact-grounded prescriptions for follow-up designs that can measure validator utility.

II. RELATED WORK

The literature relevant to validator utility in GVR-style NetOps pipelines falls into three complementary strands: (i) architectural proposals and demonstrations of GVR or verifier-interposed loops; (ii) controlled benchmark efforts that surface real-world agent failure modes relevant to NetOps; and (iii) system- and survey-level reports advocating verifier-guided orchestration and measurement-aware evaluation.

Architectural proposals and prototype demonstrations have established GVR as a practical pattern for reducing stochastic generator errors and for enabling deterministic acceptance criteria. Work advocating generate–verify–repair pipelines argues that interposing mechanistic checks (compilation, tests, policy assertions) converts some classes of stochastic generator failure into mechanically detectable events that can be rejected or repaired [1]. These demonstrations—largely in code-generation and controlled system settings—show GVR can detect structural violations and support iterative repair feedback, but they focus on defect classes that are mechanically decidable. That scope matters: validators that are powerful for syntactic/structural defects may still miss semantic intent misalignment or operator-expectation mismatches that are not encoded in mechanistic constraints.

Benchmarks and task suites in the AIOps/NetOps space document the kinds of operational failures that motivate verifier insertion. Recent benchmark work emphasizes ticket noise, false premises, and wrong-but-plausible outputs that complicate end-to-end automation; controlled suites have been used to measure agent brittleness and the practical costs of

erroneous tool calls [2],[3]. These benchmark studies make two points relevant here: first, end-to-end agentic behavior depends heavily on task realism (noisy tickets, ambiguous intent), and second, evaluation protocols and scoring choices (LLM judges, simulators, deterministic validators) materially shape measured success. Importantly, the benchmarks cited do not provide a standardized validator-metric suite tailored to NetOps GVR flows, leaving open how to quantify validator coverage and repair value across operationally representative faults.

System reports and recent preprints advocate tool-augmented orchestrators and verifier-guided recovery as practical mitigations for silent or action-triggering failures in LLM-augmented workflows [4]. These works describe self-healing orchestrators, auditable decision traces, and verifier-feedback loops that in controlled evaluations reduce the incidence of mechanically detectable failures; however, their effectiveness depends on the validators' coverage of relevant defect classes and on orchestration reliability. Surveys of LLM applications in telecommunications and NetOps synthesize adaptation and verification trade-offs, recommending metricized evaluation of verification cost versus correctness and highlighting the need for auditable, evidence-based acceptance criteria [5].

Across these strands the consistent gap is measurement: demonstrations and benchmarks show where deterministic checks help and where generators fail, but none of the cited records supplies an established, empirically validated protocol or metric suite for estimating validator false-positive/false-negative rates, detection latency, or predictive value for operational impact in GVR flows. This gap motivated our preregistered focus on a validator-metric suite and on confirmatory estimation in a paired design. We treat prior work as architectural and motivational evidence—the literature supports the reason to measure validator utility but does not itself provide comprehensive empirical estimates of validator coverage for complex NetOps semantics—so our study was designed to produce artifact-grounded, reproducible estimates under controlled conditions and to surface practical follow-up arms that the archived infrastructure can directly enable.

III. METHODOLOGY

Preregistration and primary design. The study preregistration (AC-2026-03) fixed a paired confirmatory design executed via the `hosted_netops_gvr_v1` adapter family in `scientific_confirmatory` mode. The design choice `generation_semantics = shared_initial_candidate` and `initial_generation_calls_per_task = 1` was central: for each deterministic task index the adapter made exactly one initial generator call and that identical generated candidate (the shared initial candidate) was provided to both conditions in the pair. This preregistered semantics guarantees that any observed paired difference (guarded minus baseline) can only arise from downstream deterministic validator behaviour and any permitted repair, not from independent generator sampling differences. The preregistered primary estimand was the `paired_success_rate_difference_guarded_minus_baseline`. The

confirmatory test specified an exact two-sided McNemar test for paired binary outcomes, supplemented by paired bootstrap percentile confidence intervals (`bootstrap_resamples = 10000`, `bootstrap_seed = 1729`) as recorded in the analysis artifacts.

Model, sampling, and execution accounting. The declared model in `execution/model_configuration.json` is `"gpt-5-mini"` (`requested_model = gpt-5-mini`). The recorded model configuration also contains operational fields: `max_output_tokens = 2000`, `maximum_attempts_per_call = 1`, `provider = "openai_responses"`, `store = false`, and `reasoning_effort = "minimal"`. Crucially, `execution/model_configuration.json` records `temperature = null` (`temperature unset`). Null/unset is not evidence of deterministic provider-side sampling and does not imply `temperature = 0`; runtime provider sampling behavior may still be stochastic and is audited via the provider-call records. The execution manifest and `deterministic_reconciliation` record provider-call audit fields consistent with the preregistered single shared generation per task: `hosted_model_call_event_count = 200`, `completed_hosted_model_call_event_count = 200`, `cache_hit_count = 0`, `cache_miss_count = 200`, and `stage_counts.shared_initial_generation = 200`. Because the preregistered model-call budget permitted up to `maximum_model_calls = 400`, the actual `model_calls_used = 200` demonstrates strict adherence to the shared-initial single-sample policy (one hosted-model invocation per task index).

Task generation, constraints, and contamination controls. Tasks were drawn deterministically from `deterministic_netops_task_generator_v5` with `task_family = intent_configuration_repair_v1` and the preregistered deterministic index-rule `challenging_workflow_stress_v1`. Each task manifest encodes: `initial_state`, `expected_final_state`, an explicit natural-language intent, `allowed_fields` and `allowed_touched_fields`, `workflow_policies` (for example, `minimum_active_in_groups`, `must_be_down_for_fields`, `protected_interfaces`), a `required_command_sequence`, and a structured difficulty label (`"very_hard"/"extreme"`) that the generator used when instantiating the reference answer and task inputs. To preserve confirmatory integrity the preregistration specified deterministic contamination/exclusion rules: exact-duplicate outputs (identical SHA-256) and near-duplicate tasks (embedding cosine similarity > 0.95) were to be detected automatically and replaced deterministically by the next-index entry to maintain the planned `task_count`. The analysis artifacts show no contamination exclusions occurred (`analysis/contamination_summary.csv` empty) and the missingness summary reports `complete_pairs = 200`, consistent with the preregistered sampling plan.

Validator, scoring rule, and permitted repair policy. The deterministic validator used throughout the study was `deterministic_netops_validator_v5` (`validator_version = 5.0`). The validator implements mechanistic checks that are directly exposed in per-episode `validator_trace` fields archived for each episode: `normalized_configuration` (canonical command list), `parsed_command_count`,

required_command_count, observed_touched_field_count, violation_count and violation_codes, valid (boolean), validator_id/version, repair_applied (boolean), and (when applicable) repair_provider_trace metadata. The preregistered binary scoring rule for the confirmatory estimand was full_intent_constraint_validation, which maps directly to validator.valid (true = success, false = failure) in the archived episodes. The pipeline permitted at most one repair call per episode when the validator rejected a candidate; repair events would set repair_applied = true and populate repair_provider_trace fields (these were null for all episodes in this execution). The preregistration also deterministically specified that any repair introducing actions outside the task’s allowed_action set would be treated as a policy-violation repair and excluded from the primary success numerator, with that event logged as a policy-violation.

Execution policy, missingness, and sensitivity planning. The preregistered maximum_attempts_per_call = 1 disallowed manual retries; adapter and provider failures were logged and would lead to deterministic exclusions or to classification as technical failures per the missingness plan. Primary confirmatory analysis used only complete pairs; the preregistered sensitivity analysis treated any missing/incomplete episode as a failure (0) for guarded episodes to produce conservative bounds. The execution manifest records completed_episode_count = 400, failed_episode_count = 0, and complete_pair_count = 200; the analysis missingness artifact confirms no observed missing pairs (analysis/missingness_summary.csv). All missingness and audit logs (timestamps, HTTP codes, provider error messages) were archived for reproducibility.

Analysis executor and inferential details. Analysis used the paired_binary_analysis_v1 executor as preregistered. The executor computed the paired 2×2 contingency counts (n_11, n_10, n_01, n_00) from per-episode validator.valid labels; primary testing used the exact McNemar two-sided test appropriate for paired binary outcomes. Confidence intervals for the paired difference were constructed using a paired bootstrap percentile procedure with bootstrap_resamples = 10000 and bootstrap_seed = 1729 (these bootstrap parameters are recorded in analysis/analysis_log.jsonl). Secondary analyses that were preregistered but exploratory (for example, linear models predicting simulated operational-impact proxies from validator metric vectors) were to be treated as exploratory; multiplicity control was only to be applied to exploratory p-values where reported (Benjamini–Hochberg), while point estimates and intervals were to be reported without multiplicity adjustment otherwise.

Deterministic reconciliation and provider audit semantics. The execution bundle includes deterministic_reconciliation artefacts that reconcile task and provider-call accounting with analysis inputs; this reconciliation confirms that both baseline and guarded conditions observed the same shared_initial_candidate for each pair (cross_condition_cache_key_reuse_observed = false indicates no hidden cross-condition cache reuse beyond the single sampled generation). Provider-call audit fields

(hosted_model_call_event_count = 200, cache_miss_count = 200) plus the model_calls_used = 200 entry make explicit that one model call produced the shared_initial_candidate per task. These audit fields are central for verifying that the experimental estimand (paired difference under shared_initial_candidate) is correctly identified in the archived artifacts.

Reproducibility, archival artifacts, and permitted reanalyses. All raw responses, per-episode validator traces, provider-call traces, task manifests, model configuration JSON, analysis code, and final numeric artifacts (paired_contingency_table.csv, condition_summary.csv, analysis logs) are archived in the supplied execution bundle referenced elsewhere in this paper. The preregistered artifact set supports direct reanalysis under alternative scoring rules (for example, judging success against a looser parse-based rule), simulated fault-injection applied offline to archived reference answers, or a multi-sample re-run using the same task indices but altering initial_generation_calls_per_task; such follow-up analyses are precisely the artifact-grounded prescriptions we propose and can be preregistered and executed using the same adapter semantics. Important operational note recorded in execution/model_configuration.json: the stored temperature field is null/unset; null does not imply deterministic sampling by hosted providers and therefore does not remove the need to audit provider-call records when assessing generator variance.

IV. RESULTS

Execution accounting and primary outcomes. The adapter completed the run exactly as preregistered: planned_episode_count = 400, completed_episode_count = 400, failed_episode_count = 0, and model_calls_used = 200 (one shared initial generation per pair). Provider-call audit: hosted_model_call_event_count = 200, completed_hosted_model_call_event_count = 200, cache_hit_count = 0, cache_miss_count = 200, stage_counts.shared_initial_generation = 200. No contamination exclusions or near-duplicate exclusions occurred.

Primary confirmatory result. Every sampled shared initial candidate passed the deterministic validator and no repairs were applied in any episode. Analysis artifacts report baseline_success_count = 200/200 (success_rate = 1.0) and guarded_success_count = 200/200 (success_rate = 1.0). The paired contingency (analysis/paired_contingency_table.csv) is n_11 = 200, n_10 = 0, n_01 = 0, n_00 = 0. The paired_success_rate_difference_guarded_minus_baseline = 0.00. Exact McNemar two-sided test p-value = 1.0. Bootstrap percentile 95% confidence interval (10,000 resamples, seed = 1729) = [0.00, 0.00]. The numeric artifacts (analysis/condition_summary.csv, analysis/paired_contingency_table.csv, analysis/analysis_log.jsonl) match these reported values.

Representative validator-trace excerpt (artifact-grounded). To illustrate validator outputs and why no repairs

were triggered, we include verbatim key fields from execution/scoring/task-000004-guarded.json (pair-000004):

```
pair_id: pair-000004 normalized_configuration: "interface
edge2 admin down\ninterface edge2 vlan 40\ninterface
edge2 admin up\ninterface edge1 admin down"
parsed_command_count: 4 required_command_count: 4
observed_touched_field_count: 3 valid: true repair_applied:
false validator_id: deterministic_netops_validator_v5
validator_version: 5.0
```

Equivalent `validator_trace.validation_after.valid = true` and `repair_applied = false` hold for every archived episode; `repair_provider_trace` fields are null for all episodes. Because baseline and guarded reused the identical `shared_initial_candidate` per pair, these per-episode validator outputs explain the degenerate paired outcome: there were no validator-detected mechanical defects to correct.

Secondary and exploratory diagnostics. Per-validator FP/FN estimates are degenerate given no observed rejections; detection-latency and repair-invocation distributions are uninformative because `repair_applied = false` for all episodes. The execution bundle contains full per-episode traces and raw responses (execution/raw_results.jsonl) to support reanalysis under alternative scoring or injected-fault conditions. Contamination and missingness artifacts confirm no missing or excluded episodes (analysis/missingness_summary.csv: `complete_pairs = 200`).

Artifact-grounded diagnostic interpretation. Under `shared_initial_candidate` semantics any paired difference could only arise from validator-triggered repair. The degenerate all-valid outcome is explained by three artifact-grounded determinants visible in the archive: (1) the deterministic task generator produced candidates that satisfied the validator’s mechanistic constraints for the recorded generation seeds; (2) the single-sample-per-task sampling regime removed generator variability as a source of disagreements; (3) mechanistic validators detect structural and policy violations but can be blind to semantic intent misalignment (correct-by-structure yet incorrect-by-intent). These jointly explain why no repairs occurred and the confirmatory test could not detect the preregistered 10-percentage-point improvement target.

V. DISCUSSION

Design implications of `shared_initial_candidate` semantics. The preregistered `shared_initial_candidate` choice intentionally isolates downstream validator and repair effects: because baseline and guarded evaluate the same generator output for each task, any paired improvement must originate from validator-triggered modification or repair rather than from independent generator variance. This property makes the paired estimand a precise probe of repair utility but also concentrates a single-sample ceiling risk: if the shared draw aligns with the validator for most tasks, the design yields no observable discordant pairs. The archive documents exactly this outcome (`model_calls_used = 200`, one shared generation per pair, and `n_11 = 200`), so the null paired difference is a direct, design-consistent artifact rather than an execution anomaly.

Validator scope and a compact taxonomy. Artifact-grounded traces (`validator_id = deterministic_netops_validator_v5`, `validator_version = 5.0`) show the validator enforces mechanistic, checkable properties: syntactic/format correctness, presence of required commands (`required_command_count`), permitted-field constraints (`allowed_touched_fields`), and explicit workflow policies (e.g., `minimum_active_in_groups`). We use a pragmatic taxonomy to reason about coverage and blind spots: (A) syntactic/format checks, (B) structural/ordering constraints (required sequences), (C) policy/domain constraints (protected interfaces, group minima), and (D) semantic intent alignment (whether the produced sequence actually implements the operator intent beyond mechanical constraints). The archived outputs show high coverage for A–C under the synthetic bank and generation seeds used; category D remains the primary open detection challenge because it requires either richer intent models, supplemental semantic validators, or human adjudication.

Measurement strategies that follow directly from the archive. The artifact bundle supports concrete, experimentally controlled measurement protocols to make validator FP/FN estimable while preserving preregistration rigor. Two complementary, artifact-grounded strategies are: 1) Deterministic fault-injection (recommended preregistered arm): inject known, labeled faults into a controlled fraction of tasks (syntactic mistakes, omitted required commands, swapped intent labels, or policy violations) so that baseline invalidity is non-zero. Because injections are deterministic and recorded in task payloads, the experimenter can compute validator true-positive and false-negative rates against the injected ground truth without human adjudication. This is consistent with our earlier prescription and is supported by the stored `task_manifest` entries and generator seeds. 2) Multi-sample reintroduction of generator variance: adopt `initial_generation_calls_per_task = k >= 3` independent draws per task with recorded seeds. Allow the guarded pipeline to accept any validator-approved draw or to perform repair on a rejected draw. This reintroduces generator-side variability and produces observable discordant pairs when some draws fail validator checks while others pass. The present archive shows that removing generator variance (`k = 1` shared draw) can mask potential repair utility; switching to multi-draw sampling is a direct remedy. Both strategies are implementable within the same `hosted_netops_gvr_v1` adapter semantics and are therefore immediately reproducible from the archived infrastructure.

Statistical and inferential considerations. The paired design isolates repair effects but relies on baseline invalidity or generator variance to provide statistical power for detecting repair utility. Our preregistered power target (detecting a 0.10 absolute paired improvement at ~80% power) assumed a non-negligible baseline failure rate; the observed ceiling (`baseline_success_rate = 1.0`) violates that assumption and renders the primary test uninformative. Artifact-grounded remedies include increasing sample size only when baseline failure probability is non-zero, or altering the design to ensure a controlled non-zero baseline invalidity (for example by

deterministic fault injection). Additionally, using multi-draw sampling reintroduces within-pair discordance and increases the potential for detecting repair benefits without requiring impractically large n .

Design trade-offs in validator strictness and operational cost. Mechanistic validators that are stricter on structural or policy checks reduce the risk of silent mechanical errors but may increase false positives (rejecting acceptable variants) and thus trigger more repairs or human review. Conversely, permissive validators reduce repair invocation but risk undetected defects. The appropriate operating point depends on operational cost trade-offs (verification cost, human review burden, repair latency). While our execution did not observe repairs and thus cannot measure these trade-offs empirically, the execution artifacts (including per-episode `parsed_command_count`, `required_command_count`, and `repair_applied` flags) form the necessary raw data fields for future experiments that instrument and quantify repair cost and human-review burden under alternative validator thresholds.

Practical orchestration and human-in-the-loop designs. The archive suggests pragmatic deployment patterns to mitigate semantic-misalignment risk (category D above) even when mechanistic validators perform well: (1) multi-validator arbitration (cross-checks using two independent validators with different coverage profiles), (2) gated human-in-the-loop acceptance for changes flagged as semantically risky, and (3) auditable decision traces that preserve the initial candidate, validator results, and any repair prompts/outputs for operator review. These are not measured outcomes in this run but are operationally plausible mitigations that can be evaluated empirically using the archived adapter and task bank.

How the archived artifacts support reproducible next steps. All claims above are anchored to explicit artifacts in the bundle: `task_manifest` entries (deterministic task generator seeds and required sequences), `execution/model_configuration.json` (declared model = `gpt-5-mini` and temperature = null), per-episode validator traces (`validation_before/after`, valid flags), and analysis artifacts (`paired_contingency_table.csv`, `condition_summary.csv`). Because these artifacts encode both the deterministic seeds and the full validator observations, they permit exact replication of the present ceiling outcome and straightforward extensions (inject faults, increase k , add validators) that preserve preregistration and reproducibility.

Threats to validity (brief restatement). Internal validity: the `shared_initial_candidate` semantics intentionally isolate repair effects but increase single-sample ceiling risk. Construct validity: `validator.valid` operationalizes mechanistic correctness but may not capture semantic intent misalignment. External validity: synthetic tasks, a single model (`gpt-5-mini`), and a single deterministic validator limit generalization to heterogeneous production networks and alternative model/validator designs. Conclusion validity: the degenerate outcome produces no variance for the primary estimand, so inferential conclusions about validator benefit are not possible from this execution alone. These limitations are explicit in the archived manifests and motivate the artifact-grounded follow-up arms described

above.

VI. LIMITATIONS

- `Shared_initial_candidate` semantics constrained inference: baseline and guarded reused the same initial candidate per pair; any paired difference could only arise from validator-triggered repair (not from independent generator sampling).
- Synthetic task bank (difficulty 'very_hard'/'extreme') is not equivalent to heterogeneous production networks (multi-vendor devices, real ticket noise); external validity is limited.
- Single-model experiment: only `gpt-5-mini` was executed; no multi-model generality claims are made.
- No repairs were applied because all initial candidates passed the deterministic validator; therefore the study could not estimate repair effectiveness or validator false-negative behavior in this execution.
- Validator scope: `deterministic_netops_validator_v5` enforces mechanistic intent-constraint checks but does not guarantee detection of semantic intent misalignment (correct-by-structure but incorrect-by-intent).
- Recorded `execution/model_configuration.json` temperature = null/unset does not imply deterministic sampling; provider-side sampling behavior may vary and is documented in execution manifests.

VII. CONCLUSION

We executed a fully preregistered paired confirmatory study (AC-2026-03) under `shared_initial_candidate` semantics to measure validator utility in NetOps GVR pipelines. For the synthetic task bank, the sampled `gpt-5-mini` generations, and `deterministic_netops_validator_v5` used here, every sampled initial candidate passed the validator's mechanistic checks so no repairs were applied and guarded success equaled baseline (200/200). The preregistered power assumptions (targeted 10-percentage-point improvement) were invalidated by this ceiling outcome. The result is artifact-grounded and reproducible from the archived bundle. We publish the exact execution and analysis artifacts to support replication and provide concrete, preregistered experiment prescriptions (fault injection, multi-sample generation, multi-validator comparison) that are required to produce estimable validator FP/FN behavior and repair value in operationally meaningful conditions.

DISCLOSURE STATEMENT

Disclosure (concise): Declared model and provider: `gpt-5-mini` via provider bundle '`openai_responses`'. Orchestration/adapter: `hosted_netops_gvr_v1`. Deterministic validator: `deterministic_netops_validator_v5` (`validator_version` = 5.0). Preregistration: AC-2026-03 (`preregistration_sha256` = `de37b485421cb0895a98df38b6b3baad1dc7c13c81065e3d8240d2a94a0ff844`). Master prompt immutable reference: `provenance/master_prompt.txt`, SHA-256 = `1872df1e1805d2d96940456ca016bd665d1d5196add77f5acd1f582bb39b15ba`. Autonomy boundary: a human Research Director

selected the broad topic family and approved the master prompt before the autonomy lock; after the lock the AI system autonomously performed literature review, preregistration, experiment design, execution, analysis, internal peer review and manuscript drafting without manual editing of technical content. Sampling/generation semantics: `shared_initial_candidate` (one initial sampled generation per task reused for baseline and guarded); `execution/model_configuration.json` recorded `temperature = null/unset` (`null/unset` is not evidence of deterministic sampling and does not imply `temperature = 0`). Call budget and usage (preregistered): `maximum_model_calls = 400`; `actual_model_calls_used = 200` (one shared initial generation per pair). All raw outputs, per-episode validator traces, execution manifests, and analysis artifacts are archived in the supplied execution bundle (see `execution/execution_manifest.json`). No human manually edited the manuscript technical content after the autonomy lock.

REFERENCES

- [1] [1] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments," SSRN Electronic Journal, 2026. doi:10.2139/ssrn.6754899.
- [2] [2] K.-H. Tseng, N. Bogahawatta, Y. Ginige, K. Patel, K. Dakic, S. Seneviratne, "Fault-Bench: Towards Benchmarking Network Troubleshooting LLM Agents under Unreliable User Tickets," arXiv:2608.27021, 2026.
- [3] [3] Y. Liu, C. Pei, L. Xu, B. Chen, M. Sun, Z. Zhang, Y. Sun, S. Zhang, K. Wang, H. Zhang, J. Li, G. Xie, X. Wen, X. Nie, M. Ma, P. Dan, "OpsEval: A Comprehensive IT Operations Benchmark Suite for Large Language Models," arXiv:2310.07637, 2023.
- [4] [4] R. S. Babu and A. Agrawal, "Self-Healing Agentic Orchestrators for Reliable Tool-Augmented Large Language Model Systems," arXiv:2606.01416, 2026.
- [5] [5] H. Zhou, C. Hu, Y. Yuan, Y. Cui, Y. Jin, C. Chen, H. Wu, D. Yuan, L. Jiang, D. Wu, X. Liu, J. Zhang, X. Wang, J. Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities," IEEE Commun. Surveys & Tutorials, 2024. doi:10.1109/COMST.2024.3465447.